```
function [retmap,retvisited,retsteps] = astar(mapfile,startlocation,targetlocation)

% Convert the map file into a MATLAB matrix
map = map_convert(mapfile);
retmap = map;

% Get dimensions of the map
[maprows,mapcols] = size(map);

% gScore = cost from start to each cell
gScore = inf(maprows,mapcols);

% fScore = estimated total cost from start to target
fScore = inf(maprows,mapcols);

% Keep track of visited cells
visited = false(maprows,mapcols);

% Keep track of the parent of every cell
parent = zeros(maprows,mapcols,2);

% Distance from start to itself is zero
gScore(startlocation(1),startlocation(2)) = 0;

% Heuristic estimate from start to target
fScore(startlocation(1),startlocation(2)) = ...
    abs(startlocation(1) - targetlocation(1)) + ...
    abs(startlocation(2) - targetlocation(2));

% Open set
% Contains cells that still need to be considered
openSet = startlocation;

% Store order in which cells are visited
retvisited = [];

% Directions:
%       up
%       down
%       left
%       right
directions = [
    -1  0
     1  0
     0 -1
     0  1
];

% A* algorithm
while ~isempty(openSet)

    % Find the cell in openSet with the smallest fScore
    bestIndex = 1;
    bestScore = inf;

    for i = 1:size(openSet,1)

        row = openSet(i,1);
        col = openSet(i,2);

        if fScore(row,col) < bestScore
```

```
            bestScore = fScore(row,col);
            bestIndex = i;
        end
    end

    % Get the cell with the smallest fScore
    current = openSet(bestIndex,:);

    % Remove current cell from openSet
    openSet(bestIndex,:) = [];

    % If it has already been visited, skip it
    if visited(current(1),current(2))
        continue;
    end

    % Mark current cell as visited
    visited(current(1),current(2)) = true;

    % Record the visited cell
    retvisited = [retvisited; current];

    % Have we reached the target?
    if current(1) == targetlocation(1) && ...
       current(2) == targetlocation(2)

        break;
    end

    % Check each neighbouring cell
    for k = 1:4

        neighbour = current + directions(k,:);

        row = neighbour(1);
        col = neighbour(2);

        % Check that neighbour is inside the map
        if row >= 1 && row <= maprows && ...
           col >= 1 && col <= mapcols

            % Check that neighbour is not a wall
            if map(row,col) == 0

                % Ignore already visited cells
                if ~visited(row,col)

                    % Cost of moving to neighbouring cell
                    movementCost = 1;

                    % Calculate new gScore
                    newGScore = ...
                        gScore(current(1),current(2)) + movementCost;

                    % If this is a better route
                    if newGScore < gScore(row,col)

                        % Store the new cost
                        gScore(row,col) = newGScore;

                        % Store the parent
```

```
                        parent(row,col,1) = current(1);
                        parent(row,col,2) = current(2);

                        % Calculate Manhattan heuristic
                        heuristic = ...
                            abs(row - targetlocation(1)) + ...
                            abs(col - targetlocation(2));

                        % Calculate fScore
                        fScore(row,col) = ...
                            newGScore + heuristic;

                        % Add neighbour to openSet
                        openSet = [openSet; neighbour];

                    end
                end
            end
        end
    end
end

% Reconstruct the path
retsteps = [];

% Check whether target was reached
if gScore(targetlocation(1),targetlocation(2)) ~= inf

    current = targetlocation;

    while ~(current(1) == startlocation(1) && ...
            current(2) == startlocation(2))

        % Add current cell to path
        retsteps = [current; retsteps];

        % Get parent
        parentrow = parent(current(1),current(2),1);
        parentcol = parent(current(1),current(2),2);

        current = [parentrow,parentcol];

    end

    % Add starting location
    retsteps = [startlocation; retsteps];

end

plotmap(retmap, retsteps);

end


function placestep(position,i)

% This function will plot a yellow rectangle
% and print a number in the rectangle.

position = [16-position(1) position(2)];

```

```
position = [position(2)+0.1 position(1)+0.1];

rectangle('Position',[position,0.8,0.8],'FaceColor','y');

c = sprintf('%d',i);

text(position(1)+0.2,position(2)+0.2,c,'FontSize',10);

end
```